

DAY -09

Task 01

Can you create a Maven based spring project using ide..

- Project name – Spring-welcome

App.java

```
package org.example;

public class App {

    public static void main(String[] args) {
        System.out.println("Welcome to Spring Maven Project!");
    }
}
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>spring-welcome</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

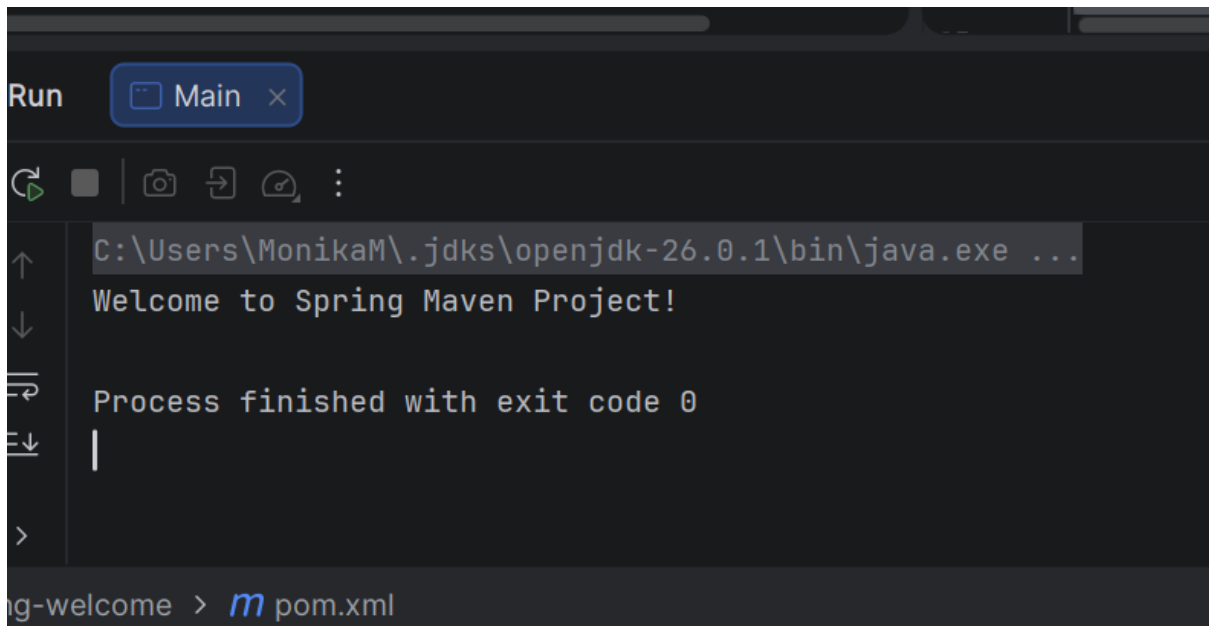
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.2.0</version>
    </dependency>
  </dependencies>

</project>
```

--- mvn clean compile → IN Terminal

--- run App.java

OUTPUT



The screenshot shows an IDE's Run console window. At the top, there's a tab labeled 'Run' with a sub-tab 'Main'. Below the tab, there's a toolbar with icons for running, debugging, and other actions. The main area of the console displays the following text: 'C:\Users\MonikaM\.jdk\openjdk-26.0.1\bin\java.exe ...', 'Welcome to Spring Maven Project!', and 'Process finished with exit code 0'. At the bottom, there's a breadcrumb trail showing 'g-welcome > m pom.xml'.

```
Run Main x
C:\Users\MonikaM\.jdk\openjdk-26.0.1\bin\java.exe ...
Welcome to Spring Maven Project!
Process finished with exit code 0
g-welcome > m pom.xml
```

Task 02:

Now create a POJO class for student so that can be managed as a spring bean

- Project name → SpringStudentDemo

App.java

```
package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class App {

    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        Student student = context.getBean(Student.class);

        student.setId(101);
        student.setName("Monika M");
        student.setCourse("Spring Framework");

        System.out.println(student);
    }
}
```

Student.java

```
package com.example;

import org.springframework.stereotype.Component;
```

```

@Component
public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
        System.out.println("Student Bean Created");
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    @Override
    public String toString() {
        return "Student{" +
            "id=" + id +
            ", name=" + name + " +
            ", course=" + course + " +
            "}";
    }
}

```

AppConfig.java

```

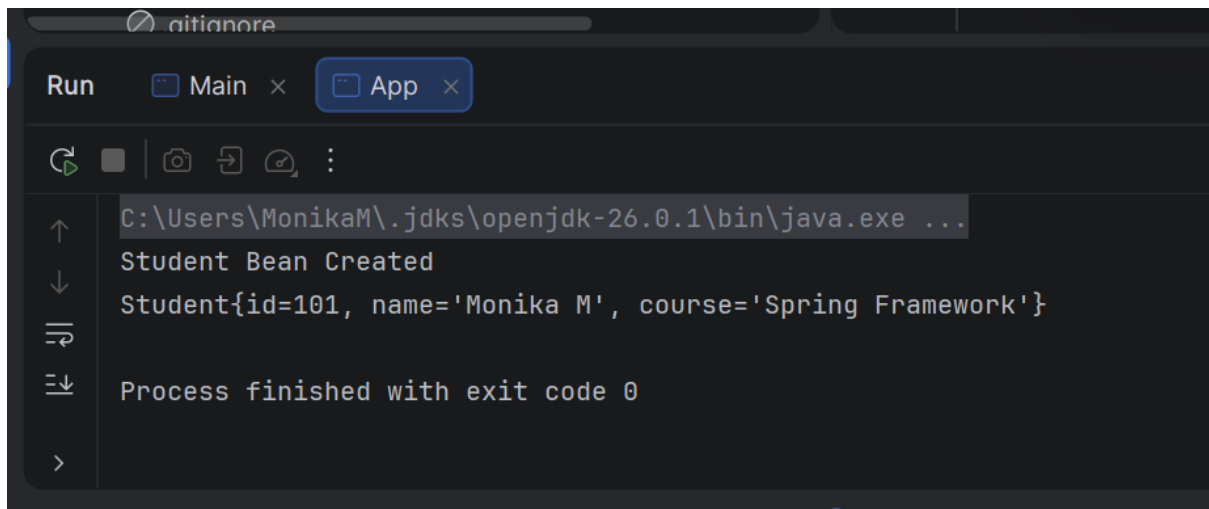
package com.example;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.example")
public class AppConfig {
}

```

OUTPUT



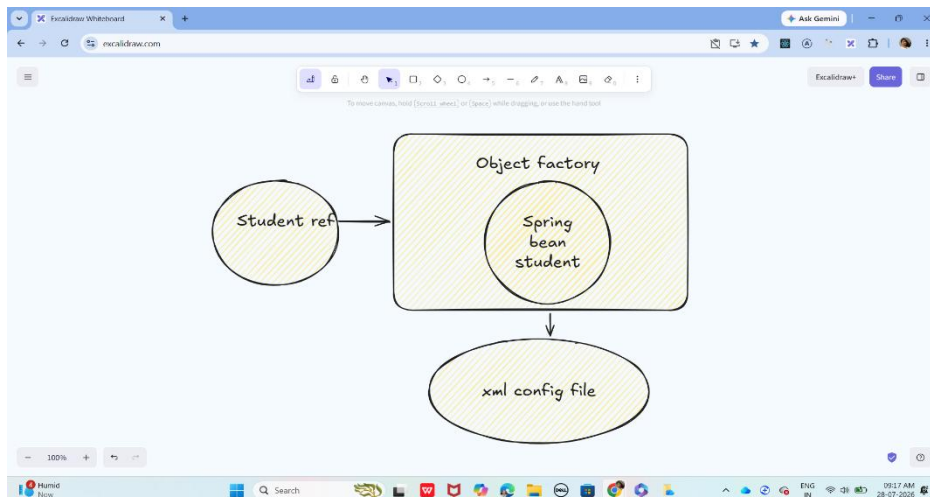
```
Run Main x App x
C:\Users\MonikaM\jdk\openjdk-26.0.1\bin\java.exe ...
Student Bean Created
Student{id=101, name='Monika M', course='Spring Framework'}
Process finished with exit code 0
```

Task 03:

Configure the above Student bean in the *bean-factory-demo.xml* file.

That means you need to configure the Student bean in the XML file so the BeanFactory knows how to create it.

And create a main class and run it.. A pictorial rep of the scenario is given below



Main.java

```
package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {
```

```

public static void main(String[] args) {

    ApplicationContext context =
        new ClassPathXmlApplicationContext("bean-factory-demo.xml");

    Student student = (Student) context.getBean("student");

    System.out.println(student);
}
}

```

Student.java

```

package com.example;

import org.springframework.stereotype.Component;

@Component
public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
        System.out.println("Student Bean Created");
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    @Override
    public String toString() {
        return "Student{" +
            "id=" + id +
            ", name=" + name + "\n" +
            ", course=" + course + "\n" +

```

```

    };
}
}

```

- Create this file inside **src/main/resources**.
- bean-factory-demo.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="student"
          class="com.example.Student">

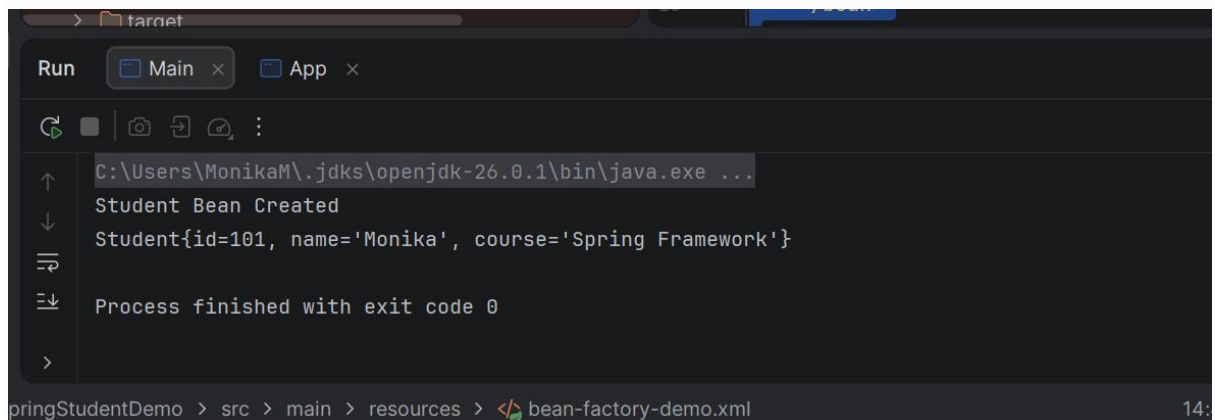
        <property name="id" value="101"/>
        <property name="name" value="Monika"/>
        <property name="course" value="Spring Framework"/>

    </bean>

</beans>

```

OUTPUT



```

Run Main x App x
C:\Users\MonikaM\jdk-26.0.1\bin\java.exe ...
Student Bean Created
Student{id=101, name='Monika', course='Spring Framework'}
Process finished with exit code 0
springStudentDemo > src > main > resources > bean-factory-demo.xml 14:

```

Task 04:

Can you explain application context with the syntax..

- ApplicationContext is a Spring container that creates and manages Spring beans.
- It reads the configuration from XML, Java classes, or annotations.
- It performs dependency injection and manages the bean lifecycle.
- It is an advanced version of BeanFactory with additional features.
- Beans can be retrieved using the `getBean()` method after loading the configuration.

Task 05:

Why use application context .. plz list some uses?

- Creates and manages Spring beans automatically.

- Performs dependency injection to provide required objects.
- Loads configuration information from XML files, Java classes, or annotations.
- Manages the complete lifecycle of beans from creation to destruction.
- Provides the `getBean()` method to access Spring-managed objects.
- Supports event handling between different components of the application.

Task 06

What are features of Spring Framework .. list them with 1 or 2 lines explanation?

- **Dependency Injection:** Dependency Injection is a design pattern where the Spring container automatically provides the required dependencies to a class, instead of the class creating them itself. This promotes loose coupling, easier testing and better maintainability by decoupling the object creation and usage.
- **Aspect-Oriented Programming (AOP):** AOP allows developers to separate cross-cutting concerns (such as logging, security and transaction management) from the business logic.
- **Transaction Management:** Spring provides a consistent abstraction for managing transactions across various databases and message services.

Task 07

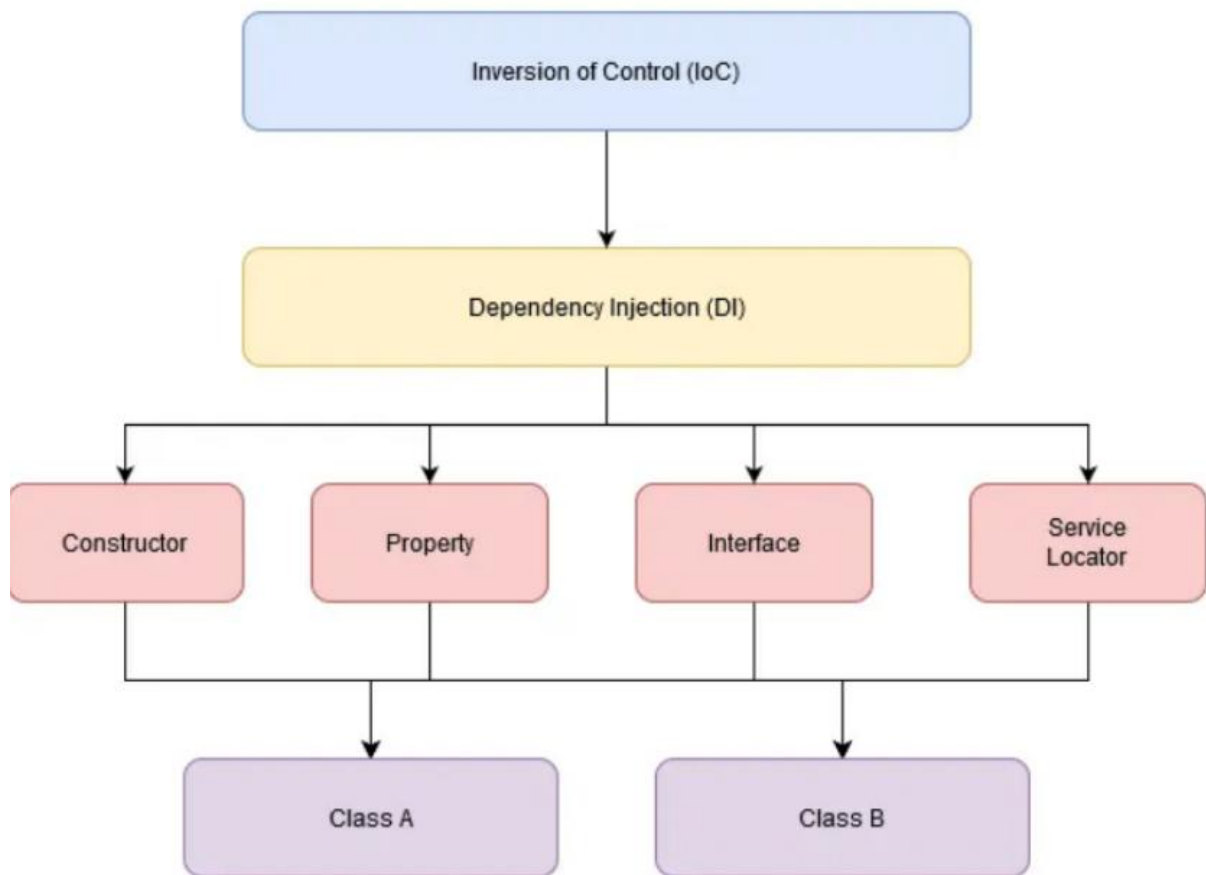
Why do you think we use spring framework.. Advantages list them down?

- Supports Dependency Injection, which makes applications loosely coupled and easier to maintain.
- Provides Inversion of Control (IoC) to manage object creation and dependencies.
- Improves code reusability by providing various modules and components.
- Provides easy integration with other technologies like JDBC, Hibernate, JPA, and web services.
- Supports MVC architecture for developing well-structured web applications.
- Provides transaction management to handle database transactions easily.
- Improves application security using Spring Security features.
- Supports easy testing because of loose coupling and dependency injection.

Task 08

What do you understand by IOC? Explain with diagram..

- Inversion of Control (IoC) is the core component of Spring that creates, configures, manages, and injects beans throughout the application lifecycle using `BeanFactory` or `ApplicationContext`. It centralizes object lifecycle management, reducing manual configuration and simplifying application development.



- **IoC (Inversion of Control)** - Transfers object creation and management responsibility from the application to the Spring container.
- **Dependency Injection (DI)** - A mechanism through which Spring provides required dependencies to objects automatically.
- **Constructor Injection** - Dependencies are provided through the class constructor.
- **Property/Setter Injection** - Dependencies are provided using setter methods.
- **Interface Injection** - Dependencies are provided through an interface (less commonly used in Spring).
- **Service Locator** - Objects obtain required services from a central service provider.
- **Class A and Class B** - These are application classes whose objects are created and managed by the Spring container.

Task 09:

How many types of IOC do you have in spring?

What are they with explanation

- **Types of IoC in Spring**

There are mainly two types of IoC in Spring:

1. **Constructor Injection**

- In constructor injection, the Spring container provides the required dependency through the constructor of a class.
- It is used when an object requires its dependencies at the time of creation.
- It helps to create immutable objects and ensures that required dependencies are always available.

2. Setter (Property) Injection

- In setter injection, the Spring container provides dependencies using setter methods.
- The object is first created, and then the required dependencies are injected.
- It is useful when dependencies are optional or can be changed during the object lifecycle.

Task 10:

What is DI in spring?

- **Dependency Injection** is the mechanism through which the Spring container supplies required dependencies to an object instead of the object creating them itself. It promotes loose coupling, improves code reusability, and makes applications easier to test and maintain.

Task 11:

How many types of DI do we have?

→ There are **two main types of Dependency Injection (DI) in Spring:**

1. Constructor Injection

- Dependencies are provided through the constructor of a class.
- It is used when dependencies are mandatory.

2. Setter (Property) Injection

- Dependencies are provided using setter methods.
- It is used when dependencies are optional or can be changed later.

Task 12:

What is Spring Application Configuration?

- Spring Application Configuration is the process of defining and managing the settings required for a Spring application, such as bean creation, dependencies, and application behaviour

- It tells the Spring container how to create, configure, and manage objects (beans). Spring application configuration can be done using XML configuration, Java-based configuration, or annotations.

Task 13

What are the different ways to configure spring applications?

- **XML-Based Configuration**-In this method, Spring beans and their dependencies are configured in an XML file.
Example: applicationContext.xml or bean-factory-demo.xml.
- **Annotation-Based Configuration** - In this method, annotations are used to configure Spring beans.
Common annotations are @Component, @Autowired, @Service, and @Repository.
- **Java-Based Configuration** - In this method, Java classes are used to configure beans instead of XML files.
It uses annotations like @Configuration, @Bean, and @ComponentScan.
- **Spring Boot Configuration** - Spring Boot uses auto-configuration and properties files to configure applications easily.
Common files are application.properties and application.yml.

Task 14:

Why we need spring application configuration?

- Spring Application Configuration is needed to define and manage beans, dependencies, and settings of a Spring application.
- It helps the Spring container create, configure, and manage objects automatically, reducing manual coding and improving maintainability.

Task 15

What are the difference ways to configure spring application?

- Same as Task13

Task 16

What is bean scope .. give different scopes with description

- Bean Scope defines the lifecycle and number of instances of a Spring bean created by the container.
- The main scopes are Singleton (one instance per container) and Prototype (new instance every time requested).
- Request, Session, Application, and WebSocket scopes are used mainly in web applications.

Task 17:

What is autowiring in spring?

- Autowiring in Spring is a feature that automatically injects the required dependencies into a bean without manually configuring them. It reduces the need for explicit dependency configuration and is commonly done using the `@Autowired` annotation.

Task 18:

What are diff ways of autowiring?

- By Name – Injects dependency by matching the bean name with the property name.
- By Type – Injects dependency by matching the data type of the property.
- By Constructor – Injects dependency through the constructor of the class.
- Using `@Autowired` – Automatically injects dependencies using the `@Autowired` annotation.
- Using `@Qualifier` – Specifies the exact bean to inject when multiple beans of the same type exist.

Task 19

What is diff between a normal java object and Spring bean?

- A normal Java object is created and managed by the developer using the `new` keyword, and its lifecycle and dependencies are handled manually. A Spring bean is an object that is created, configured, and managed by the Spring IoC container. Spring beans support features like dependency injection, lifecycle management, and configuration, which makes application development easier and more maintainable.

Task 20

What is diff between `@component` and `@bean`

- `@Component` is used to make a class a Spring bean automatically through component scanning. It is placed directly on the class, and Spring creates an object of that class.
- `@Bean` is used to manually define a bean inside a Java configuration class using a method annotated with `@Bean`. It gives more control over the object creation process.

Task 21

A different way to create an application using

<https://start.spring.io/> try it .. and let me know what is it?

The screenshot shows the Spring Initializr web application. The interface is divided into several sections:

- Project:** Includes radio buttons for **Gradle - Groovy** (selected), **Gradle - Kotlin**, and **Maven**.
- Language:** Includes radio buttons for **Java** (selected), **Kotlin**, and **Groovy**.
- Spring Boot:** Includes radio buttons for **4.1.1 (SNAPSHOT)**, **4.1.0** (selected), and **4.0.8 (SNAPSHOT)**, with **4.0.7** listed below.
- Project Metadata:** Includes text input fields for **Group** (com.example), **Artifact** (welcome-app), and **Package name** (com.example.welcome-app).
- Packaging:** Includes radio buttons for **Jar** (selected) and **War**.
- Configuration:** Includes radio buttons for **Properties** (selected) and **YAML**.
- Java:** Includes radio buttons for versions 26, 25, 21, and **17** (selected).
- Dependencies:** A section with the text "No dependency selected" and a button "ADD DEPENDENCIES... CTRL + B".

At the bottom, there are three buttons: **GENERATE CTRL + G**, **EXPLORE CTRL + SPACE**, and an ellipsis button.

Steps to create a Spring Boot application using Spring Initializr:

1. Open the website: <https://start.spring.io/>
2. Select project type (Maven or Gradle), language (Java), and Spring Boot version.
3. Enter project details like Group, Artifact, and project name.
4. Add required dependencies (for example: Spring Web, Spring Data JPA, MySQL Driver).
5. Click Generate to download the project as a ZIP file.
6. Extract the ZIP file and import it into an IDE like Eclipse, IntelliJ IDEA, or STS.
7. Run the main class containing `@SpringBootApplication` to start the Spring Boot application.

Task 22

Using Spring initialiser create an employee class, appconfig file, and main class and see how it works

← ↻

https://start.spring.io

🏠 ☆ ⚙️ 👤 ...

☰

🔄

Project

☒ Gradle - Groovy

☐ Gradle - Kotlin

☐ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.1 (SNAPSHOT)

☒ 4.1.0

☐ 4.0.8 (SNAPSHOT)

☐ 4.0.7

Project Metadata

Group

com.example

Artifact

employee-app

Package name

com.example.employee-app

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Java

☐ 26

☐ 25

☐ 21

☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

No dependency selected

🐙

GENERATE CTRL + G

EXPLORE CTRL + SPACE

...

- Employee.java

```
- package com.example.employeeapp;

import org.springframework.stereotype.Component;

@Component
public class Employee {

    private int id;
    private String name;
    private String department;

    public Employee() {
        System.out.println("Employee Bean Created");
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDepartment() {
        return department;
    }
}
```

```

    }

    public void setDepartment(String department) {
        this.department = department;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id +
            ", name=" + name +
            ", department=" + department + "]";
    }
}

```

AppConfig.java

```

package com.example.employeeapp;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.example.employeeapp")
public class AppConfig {
}

```

- EmployeeAppApplication.java

```

package com.example.employeeapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

@SpringBootApplication
public class EmployeeAppApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            SpringApplication.run(EmployeeAppApplication.class, args);

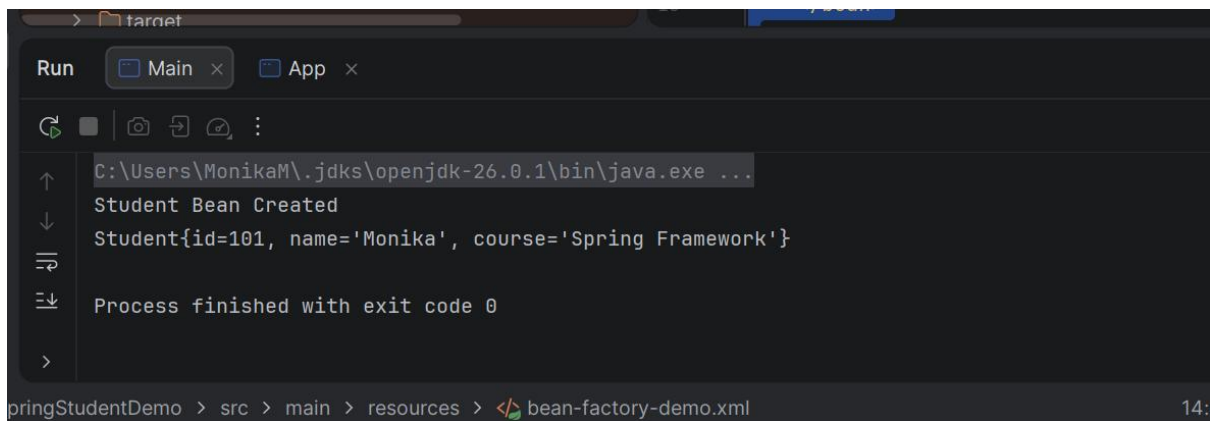
        Employee employee =
            context.getBean(Employee.class);

        employee.setId(101);
        employee.setName("Monika");
        employee.setDepartment("IT");

        System.out.println(employee);
    }
}

```

OUTPUT



```
Run Main x App x
C:\Users\MonikaM\.jdk\openjdk-26.0.1\bin\java.exe ...
Student Bean Created
Student{id=101, name='Monika', course='Spring Framework'}
Process finished with exit code 0
springStudentDemo > src > main > resources > bean-factory-demo.xml 14:
```

Task 23:

We have spring annotations:

1. `@Bean` – Used to create and register a Spring bean manually inside a `@Configuration` class.
2. `@Controller` – Used to create a Spring MVC controller that handles web requests.
3. `@Value` – Used to inject values from properties files or configuration into fields.
4. `@ComponentScan` – Used to specify the packages that Spring should scan for components.
5. `@Configuration` – Indicates that a class contains Spring bean configuration.
6. `@Service` – Used to mark a class as a service layer component containing business logic.
7. `@Repository` – Used to mark a class as a data access layer component for database operations.
8. `@Qualifier` – Used with `@Autowired` to specify a particular bean when multiple beans of the same type exist.
9. `@Required` – Used to indicate that a bean property must be set during configuration.
10. `@Component` – Marks a class as a Spring bean so that Spring can automatically detect and manage it.
11. `@Autowired` – Used to automatically inject dependencies into a Spring bean.
12. `@Scope` – Defines the lifecycle and scope of a Spring bean, such as singleton or prototype.

Task 24:

What are the features of springboot?

- ☐ Auto Configuration automatically configures the application based on project dependencies.
- ☐ Embedded Servers like Tomcat allow applications to run without external servers.
- ☐ Starter Dependencies simplify dependency management and project setup.
- ☐ Standalone Applications can be run using a simple main() method.
- ☐ Production-Ready Features such as Actuator provide monitoring, health checks, and metrics.

Task 25:

What are the advantages of Spring boot?

1. Reduces development time by minimizing configuration and boilerplate code.
2. Provides embedded servers like Tomcat, so no separate server setup is needed.
3. Simplifies dependency management using Spring Boot Starter dependencies.
4. Supports rapid development of REST APIs and microservices.
5. Offers production-ready features such as health monitoring, metrics, and logging through Actuator.

Task 26

What are the key components of Spring boot?

- ☐ Spring Boot Starters – Predefined dependencies that simplify project setup.
- ☐ Auto Configuration – Automatically configures application components based on dependencies.
- ☐ Spring Boot Actuator – Provides monitoring, health checks, and metrics.
- ☐ Embedded Servers – Includes Tomcat, Jetty, or Undertow for standalone execution.
- ☐ Spring Boot CLI – Command-line tool for developing and running Spring applications quickly.

Task 27

Why spring boot over spring?

- ☐ Less Configuration – Spring Boot reduces XML and manual configuration through Auto Configuration.
- ☐ Embedded Servers – Comes with built-in Tomcat/Jetty, so no need to deploy WAR files on external servers.
- ☐ Faster Development – Starter dependencies and default settings help build applications quickly.
- ☐ Easy Dependency Management – Spring Boot Starters automatically manage compatible library versions.
- ☐ Production Ready – Provides Actuator for monitoring, health checks, and application metrics.

Task 28:

Can you explain the internal working of spring boot?

- Spring Boot internally works by starting the application through `SpringApplication.run()`, which creates and initializes the Spring Application Context (IoC container).
- The `@SpringBootApplication` annotation enables component scanning, auto-configuration, and configuration management.
- Spring Boot automatically detects required dependencies and configures beans such as controllers, services, databases, and the embedded Tomcat server.
- It scans the project for Spring components, creates and manages their objects, and handles dependency injection. Once the embedded server starts, incoming requests are routed through the `DispatcherServlet` to the appropriate controller methods, which process the request and return the response.

Task 29:

List out the starter dependencies for spring boot

- ☐ **spring-boot-starter-web** – For building web applications and REST APIs.
- ☐ **spring-boot-starter-data-jpa** – For database access using JPA and Hibernate.
- ☐ **spring-boot-starter-security** – For authentication and authorization.
- ☐ **spring-boot-starter-test** – For unit and integration testing.
- ☐ **spring-boot-starter-thymeleaf** – For building web applications with Thymeleaf templates.
- ☐ **spring-boot-starter-jdbc** – For JDBC database connectivity.
- ☐ **spring-boot-starter-validation** – For validating user input using Bean Validation.
- ☐ **spring-boot-starter-actuator** – For monitoring, metrics, and health checks.
- ☐ **spring-boot-starter-mail** – For sending emails.

Task 30:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class MyApplication
{
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

In this above code ..

What is the internal working of above code ..

Give thhe step by step explanation

-
1. JVM starts the application and executes the `main()` method.
 2. `SpringApplication.run()` is called.

3. Spring Boot creates the Application Context (IoC Container).
4. `@SpringBootApplication` is processed.
5. `@ComponentScan` scans packages for Spring components.
6. Spring creates and registers all detected beans.
7. `@EnableAutoConfiguration` automatically configures required components based on dependencies.
8. Dependency Injection is performed for all required beans.
9. Embedded Tomcat (or configured server) starts.
10. The application context is fully initialized.
11. Application becomes ready to accept requests.
12. Incoming requests are handled by `DispatcherServlet`.
13. `DispatcherServlet` routes requests to the appropriate Controller.
14. Controller processes the request (optionally using Service and Repository layers).
15. Response is returned to the client.

Task 31:

What is the syntax to change the port of the embedded Tomcat server in Spring Boot?

- `mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=9090 --name=Monika"`

- `mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=9090"`